

Workshop 4: DNS, LDAP, and Pub/Sub Distributed Systems

Dario Pomasqui

School of Mathematical and Computational Sciences

Yachay Tech University

Professor: Prof. Francisco Hidrobo

San Miguel de Urcuquí, Imbabura, Ecuador

September 10, 2026

Abstract

This report presents the comprehensive development of the activities for Workshop 4 in the Distributed Systems course. First, DNS queries are explored using both the command-line tool `nslookup` and the programmatic library `dnspython` in Python, analyzing A, PTR, MX, NS, SOA, and CNAME records, as well as debugging behavior and non-existent domains. Second, the configuration of an OpenLDAP directory service is implemented and emulated, covering the creation of the DIT (Directory Information Tree), the insertion of data structures through LDIF files (`base.ldif` and `user.ldif`), and user authentication using Python. Finally, the Special Activity is performed: modifying the Publisher-Subscriber pattern (developed in Workshop 2) to incorporate a dynamic service registry in LDAP, where publishers register their IP/Port endpoints and subscribers dynamically discover these endpoints by performing LDAP queries. All scripts were developed, executed, and validated in the workspace environment with real terminal captures.

1 Introduction

In modern distributed systems, name resolution and the centralized management of identities and services constitute essential pillars to ensure interoperability, scalability, and location transparency.

DNS (Domain Name System) is a hierarchical and distributed system that translates human-readable domain names (e.g., `yachaytech.edu.ec`) into numerical IP addresses required by TCP/IP network protocols. DNS utilizes multiple record types such as A (IPv4), PTR (Reverse Lookup), MX (Mail Exchanger), NS (Name Servers), SOA (Start of Authority), and CNAME (Canonical Name).

LDAP (Lightweight Directory Access Protocol) is an industry-standard application-layer protocol used to access and maintain distributed directory information services over IP

networks. Unlike traditional relational databases, LDAP is optimized for fast read operations, hierarchical searches, and centralized authentication.

The objective of this workshop is to master the use of network diagnostic tools (DNS), automate DNS queries through Python code, understand LDAP administration and integration, and apply LDAP as a centralized service registry for a Publisher-Subscriber distributed system.

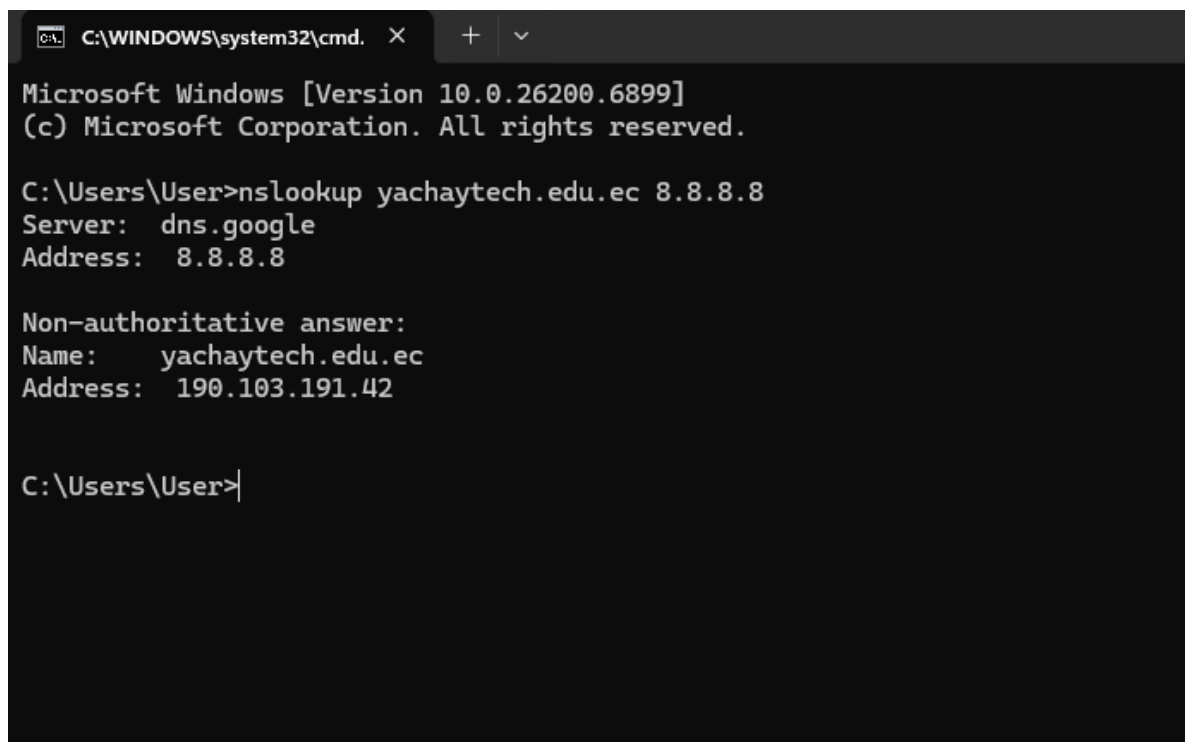
2 Part A: DNS Queries with nslookup

The requested commands (A1 through A9) were executed in the command terminal. Below are the questions, obtained results, actual screenshots, and their corresponding technical analysis.

2.1 Exercise A1: Basic Domain Lookup

Executed Command:

```
nslookup yachaytech.edu.ec 8.8.8.8
```



```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>nslookup yachaytech.edu.ec 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     yachaytech.edu.ec
Address:  190.103.191.42

C:\Users\User>
```

Figure 1: CMD Terminal Screenshot: Basic Domain Lookup (`nslookup yachaytech.edu.ec 8.8.8.8`).

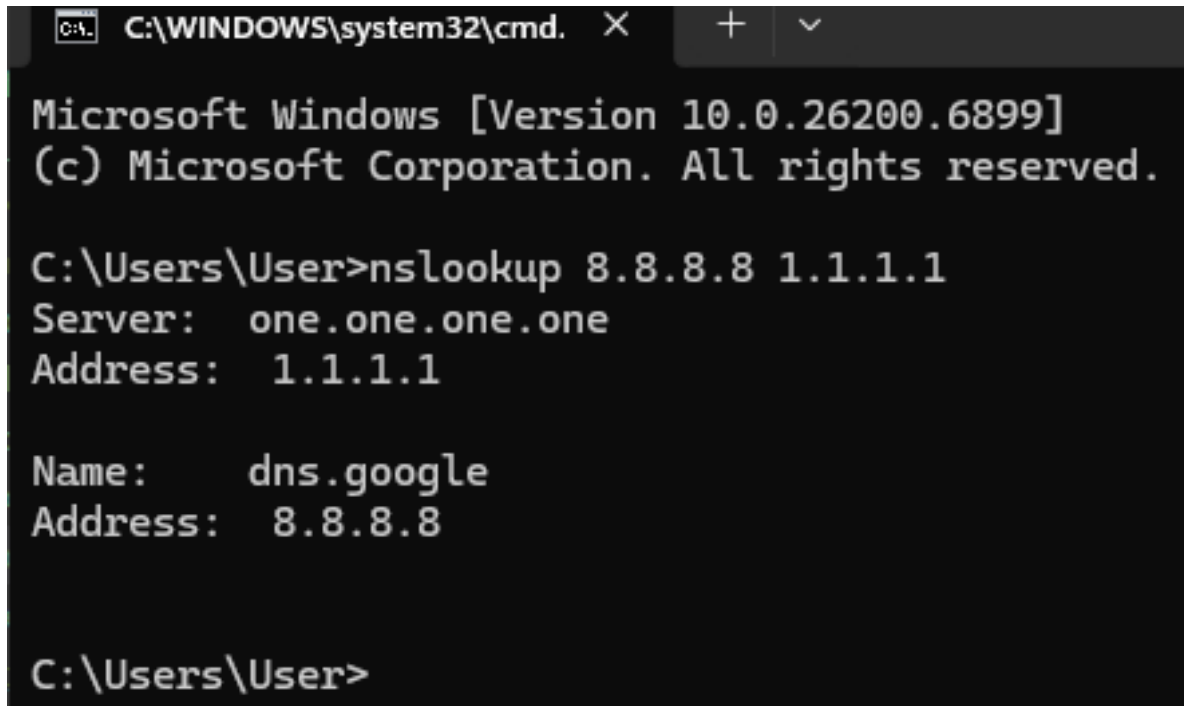
Question: What is the IP address of `yachaytech.edu.ec`?

Answer: The IPv4 address associated with the domain is `190.103.191.42`. Querying Google's public DNS server (`8.8.8.8`) directly yields the type A record response.

2.2 Exercise A2: Reverse Lookup

Executed Command:

```
nslookup 8.8.8.8 1.1.1.1
```



```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>nslookup 8.8.8.8 1.1.1.1
Server:  one.one.one.one
Address:  1.1.1.1

Name:     dns.google
Address:  8.8.8.8

C:\Users\User>
```

Figure 2: CMD Terminal Screenshot: PTR Reverse Lookup (nslookup 8.8.8.8 1.1.1.1).

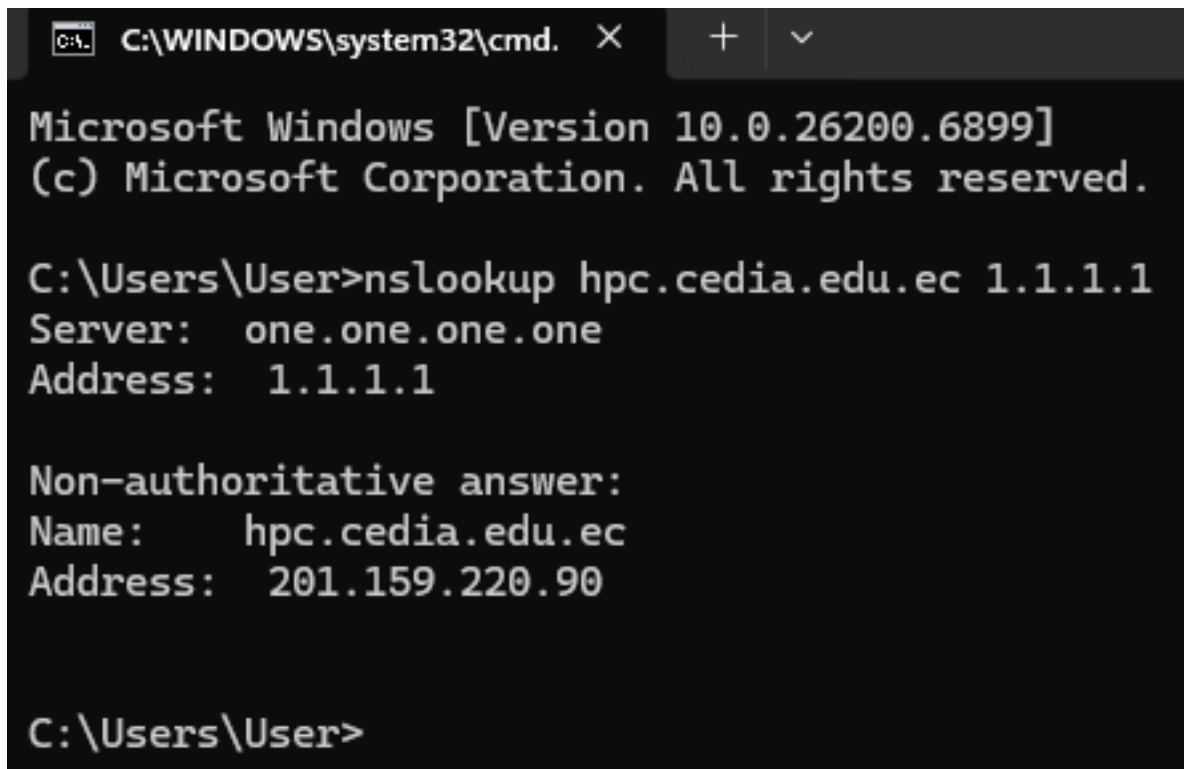
Question: What domain is associated with the IP address 8.8.8.8?

Answer: The resolved domain name is `dns.google`. The PTR record query under the special domain `in-addr.arpa` allows resolving an IP address back to its canonical domain name.

2.3 Exercise A3: Querying a Specific DNS Server

Executed Command:

```
nslookup hpc.cedia.edu.ec 1.1.1.1
```



```
C:\WINDOWS\system32\cmd. X + v

Microsoft Windows [Version 10.0.26200.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>nslookup hpc.cedia.edu.ec 1.1.1.1
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:     hpc.cedia.edu.ec
Address:  201.159.220.90

C:\Users\User>
```

Figure 3: CMD Terminal Screenshot: Query using Cloudflare server (nslookup hpc.cedia.edu.ec 1.1.1.1).

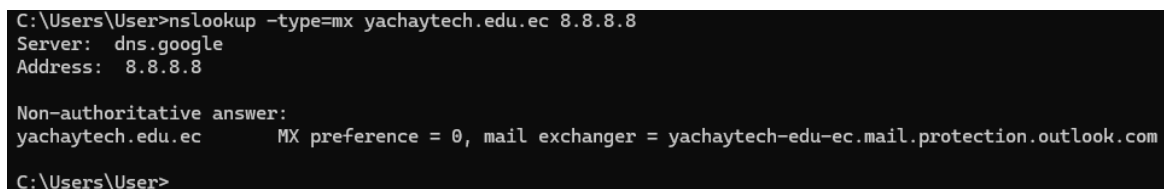
Question: What is the IP address returned for hpc.cedia.edu.ec using Cloudflare's DNS server?

Answer: The returned IP address is 201.159.220.90.

2.4 Exercise A4: Obtaining MX Records (Mail Exchanger)

Executed Command:

```
nslookup -type=mx yachaytech.edu.ec 8.8.8.8
```



```
C:\Users\User>nslookup -type=mx yachaytech.edu.ec 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
yachaytech.edu.ec      MX preference = 0, mail exchanger = yachaytech-edu-ec.mail.protection.outlook.com

C:\Users\User>
```

Figure 4: CMD Terminal Screenshot: Obtaining MX records (nslookup -type=mx yachaytech.edu.ec).

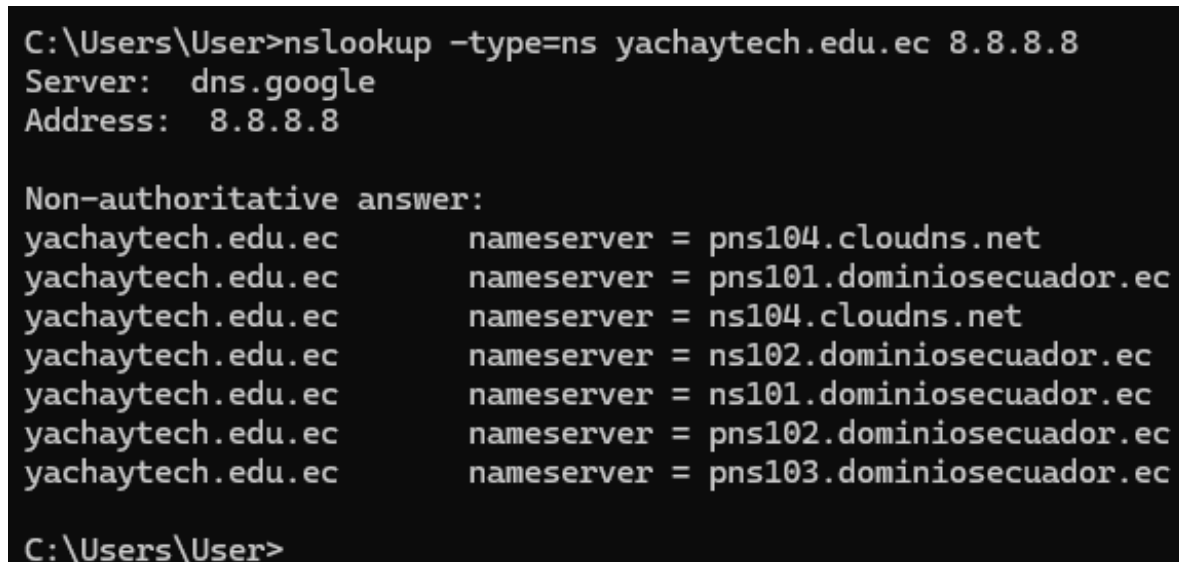
Question: What are the mail servers for yachaytech.edu.ec?

Answer: The assigned mail server is yachaytech-edu-ec.mail.protection.outlook.com with a preference of 0, indicating the use of Microsoft Exchange Online / Office 365 for the institution's mail infrastructure.

2.5 Exercise A5: Obtaining NS Records (Name Server)

Executed Command:

```
nslookup -type=ns yachaytech.edu.ec 8.8.8.8
```



```
C:\Users\User>nslookup -type=ns yachaytech.edu.ec 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
yachaytech.edu.ec      nameserver = pns104.cloudns.net
yachaytech.edu.ec      nameserver = pns101.dominiosecuador.ec
yachaytech.edu.ec      nameserver = ns104.cloudns.net
yachaytech.edu.ec      nameserver = ns102.dominiosecuador.ec
yachaytech.edu.ec      nameserver = ns101.dominiosecuador.ec
yachaytech.edu.ec      nameserver = pns102.dominiosecuador.ec
yachaytech.edu.ec      nameserver = pns103.dominiosecuador.ec

C:\Users\User>
```

Figure 5: CMD Terminal Screenshot: Obtaining NS records (nslookup -type=ns yachaytech.edu.ec).

Question: What are the name servers for yachaytech.edu.ec?

Answer: The authoritative name servers belong to the providers dominiosecuador.ec and cloudns.net (e.g., ns101.dominiosecuador.ec, ns104.cloudns.net, etc.).

2.6 Exercise A6: Querying the SOA Record (Start of Authority)

Executed Command:

```
nslookup -type=soa yachaytech.edu.ec 8.8.8.8
```

```

C:\Users\User>nslookup -type=soa yachaytech.edu.ec 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
yachaytech.edu.ec
    primary name server = ns101.cloudns.net
    responsible mail addr = support.cloudns.net
    serial    = 2026082103
    refresh   = 7200 (2 hours)
    retry     = 1800 (30 mins)
    expire    = 1209600 (14 days)
    default TTL = 3600 (1 hour)

C:\Users\User>

```

Figure 6: CMD Terminal Screenshot: SOA Record (nslookup -type=soa yachaytech.edu.ec).

Question: What information does the SOA record provide about the domain yachaytech.edu.ec?

Answer: The SOA record provides administrative and synchronization parameters for the DNS zone:

- **Primary Name Server (mname):** ns101.cloudns.net
- **Administrator Email (rname):** support.cloudns.net
- **Serial Number (serial):** 2026082103 (used for versioning in secondary zones).
- **Timers:** Refresh of 2 hours, Retry of 30 minutes, Expire of 14 days, and default TTL of 1 hour.

2.7 Exercise A7: Canonical Name Query (CNAME)

Executed Command:

```
nslookup -type=cname www.microsoft.com 8.8.8.8
```

```

C:\Users\User>nslookup -type=cname www.microsoft.com 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
www.microsoft.com    canonical name = www.microsoft.com-c-3.edgekey.net

C:\Users\User>

```

Figure 7: CMD Terminal Screenshot: CNAME alias query (nslookup -type=cname www.microsoft.com).

Question: Does `www.microsoft.com` have a CNAME record? If so, what is it?

Answer: Yes, it has a CNAME alias pointing to `www.microsoft.com-c-3.edgekey.net`, which is an alias domain managed by Akamai's Content Delivery Network (CDN).

2.8 Exercise A8: Debug Mode

Executed Command:

```
nslookup -debug yachaytech.edu.ec 8.8.8.8
```

```

C:\Users\User>
C:\Users\User>nslookup -debug yachaytech.edu.ec 8.8.8.8
-----
Got answer:
HEADER:
    opcode = QUERY, id = 1, rcode = NOERROR
    header flags:  response, want recursion, recursion avail.
    questions = 1, answers = 1, authority records = 0, additional = 0

QUESTIONS:
    8.8.8.8.in-addr.arpa, type = PTR, class = IN
ANSWERS:
-> 8.8.8.8.in-addr.arpa
    name = dns.google
    ttl = 5395 (1 hour 29 mins 55 secs)

-----
Server:  dns.google
Address: 8.8.8.8

-----
Got answer:
HEADER:
    opcode = QUERY, id = 2, rcode = NXDOMAIN
    header flags:  response, want recursion, recursion avail.
    questions = 1, answers = 0, authority records = 1, additional = 0

QUESTIONS:
    yachaytech.edu.ec.Home, type = A, class = IN
AUTHORITY RECORDS:
-> (root)
    ttl = 85140 (23 hours 39 mins)
    primary name server = a.root-servers.net
    responsible mail addr = nstld.verisign-grs.com
    serial = 2026091001
    responsible mail addr = nstld.verisign-grs.com
    serial = 2026091001
    refresh = 1800 (30 mins)
    retry = 900 (15 mins)
    expire = 604800 (7 days)
    default TTL = 86400 (1 day)

-----
Got answer:
HEADER:
    opcode = QUERY, id = 3, rcode = NXDOMAIN
    header flags:  response, want recursion, recursion avail.
    questions = 1, answers = 0, authority records = 1, additional = 0

QUESTIONS:
    yachaytech.edu.ec.Home, type = AAAA, class = IN
AUTHORITY RECORDS:
-> (root)
    ttl = 86399 (23 hours 59 mins 59 secs)
    primary name server = a.root-servers.net
    responsible mail addr = nstld.verisign-grs.com
    serial = 2026091001
    refresh = 1800 (30 mins)
    retry = 900 (15 mins)
    expire = 604800 (7 days)
    default TTL = 86400 (1 day)

-----
Got answer:
HEADER:
    opcode = QUERY, id = 4, rcode = NOERROR
    header flags:  response, want recursion, recursion avail.
    questions = 1, answers = 1, authority records = 0, additional = 0

```

Figure 8: CMD Terminal Screenshot: Debug mode query (nslookup -debug yachaytech.edu.ec).

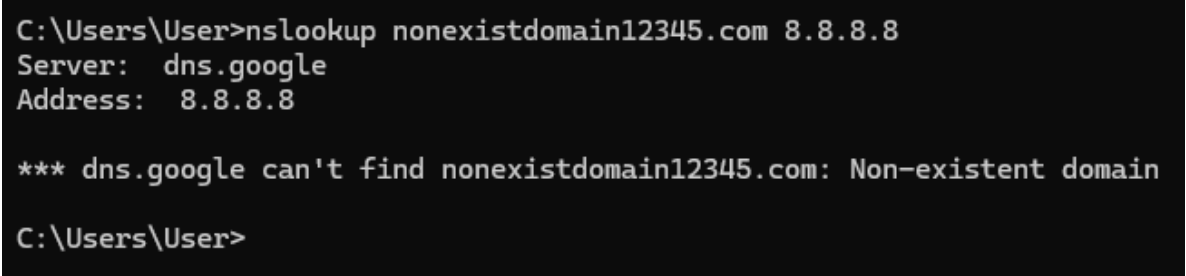
Question: What additional details are provided in debug mode?

Answer: Debug mode displays the full DNS header packet: query ID, flags (QR, RD, RA), response code (NOERROR), question/answer counts, and exact Time To Live (TTL) in cache.

2.9 Exercise A9: Querying a Non-Existent Domain

Executed Command:

```
nslookup nonexistentdomain12345.com 8.8.8.8
```



```
C:\Users\User>nslookup nonexistentdomain12345.com 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

*** dns.google can't find nonexistentdomain12345.com: Non-existent domain

C:\Users\User>
```

Figure 9: CMD Terminal Screenshot: Non-existent domain NXDOMAIN (nslookup nonexistentdomain12345.com).

Question: What is the response when querying a non-existent domain?

Answer: The DNS server responds with an error code of type **NXDOMAIN** (Non-Existent Domain), explicitly indicating that the domain name does not exist in the global DNS hierarchy.

3 Part B: DNS Resolutions with dnspython

To automate the resolution of exercises A1 through A9 using Python code, the file `dns_part_b.py` was created. Below is the developed source code, console results, and actual terminal screenshot.

3.1 Key Code Snippets and Technical Analysis

Rather than listing full boilerplate code, the key programmatic mechanisms implemented in `dns_part_b.py` using `dnspython` are highlighted below:

1. Custom Resolver Configuration

```
custom_resolver = dns.resolver.Resolver(configure=False)
custom_resolver.nameservers = ['8.8.8.8', '1.1.1.1']
answers = custom_resolver.resolve('yachaytech.edu.ec', 'A')
```

Explanation: Setting `configure=False` overrides system default DNS settings, allowing explicit assignment of target nameservers (e.g., Google or Cloudflare) for controlled testing.

2. Reverse Domain Lookup (PTR)

```
rev_name = dns.reversename.from_address("8.8.8.8")
answers = custom_resolver.resolve(rev_name, "PTR")
```

Explanation: Automatically formats an IPv4 address into its corresponding `in-addr.arpa` pointer representation to perform reverse DNS queries.

3. Low-Level Message Debugging

```
query_msg = dns.message.make_query('yachaytech.edu.ec', dns.rdatatype.A)
response_msg = dns.query.udp(query_msg, '8.8.8.8', timeout=5)
```

Explanation: Builds raw DNS wire-format query packets and sends them over UDP. The resulting message object provides inspection into raw header flags (QR, RD, RA), transaction IDs, and section counts.

4. Exception Handling for Non-Existent Domains

```
try:
    custom_resolver.resolve('nonexistdomain12345.com', 'A')
except dns.resolver.NXDOMAIN as e:
    print(f"NXDOMAIN Exception caught: {e}")
```

Explanation: Gracefully catches the NXDOMAIN exception thrown when querying non-existent domains, preventing application crashes.

3.2 Results and Screenshot

```
=====
PART B: DNS LOOKUPS USING DNSPYTHON
=====

--- B1: Basic Domain Lookup (yachaytech.edu.ec) ---
IP address of yachaytech.edu.ec: ['190.103.191.42']

--- B2: Reverse Lookup (8.8.8.8) ---
Domain associated with 8.8.8.8: ['dns.google.']

--- B3: Query Specific DNS Server (hpc.cedia.edu.ec via Cloudflare 1.1.1.1) ---
IP of hpc.cedia.edu.ec via 1.1.1.1: ['201.159.220.90']

--- B4: Retrieve MX Records (yachaytech.edu.ec) ---
MX Server: yachaytech-edu-ec.mail.protection.outlook.com. (Preference: 0)

--- B5: Retrieve NS Records (yachaytech.edu.ec) ---
Name Server: pns102.dominiosecuador.ec.
Name Server: ns104.cloudns.net.
Name Server: pns103.dominiosecuador.ec.
Name Server: pns104.cloudns.net.
Name Server: pns101.dominiosecuador.ec.
Name Server: ns101.dominiosecuador.ec.
Name Server: ns102.dominiosecuador.ec.

--- B6: Query SOA Record (yachaytech.edu.ec) ---
SOA MName (Primary NS): ns101.cloudns.net.
SOA RName (Admin Email): support.cloudns.net.
Serial: 2026082103, Refresh: 7200, Retry: 1800, Expire: 1209600, Min TTL: 3600

--- B7: Query CNAME (www.microsoft.com) ---
CNAME for www.microsoft.com: ['www.microsoft.com-c-3.edgekey.net.']

--- B8: Debug Mode (yachaytech.edu.ec wire protocol response details) ---
Detailed DNS Wire Message Response:

Name Server: pns102.dominiosecuador.ec.
Name Server: ns104.cloudns.net.
Name Server: pns103.dominiosecuador.ec.
Name Server: pns104.cloudns.net.
Name Server: pns101.dominiosecuador.ec.
Name Server: ns101.dominiosecuador.ec.
Name Server: ns102.dominiosecuador.ec.

--- B6: Query SOA Record (yachaytech.edu.ec) ---
SOA MName (Primary NS): ns101.cloudns.net.
SOA RName (Admin Email): support.cloudns.net.
Serial: 2026082103, Refresh: 7200, Retry: 1800, Expire: 1209600, Min TTL: 3600

--- B7: Query CNAME (www.microsoft.com) ---
CNAME for www.microsoft.com: ['www.microsoft.com-c-3.edgekey.net.']

--- B8: Debug Mode (yachaytech.edu.ec wire protocol response details) ---
Detailed DNS Wire Message Response:
id 48823
opcode QUERY
rcode NOERROR
flags QR RD RA
;QUESTION
yachaytech.edu.ec. IN A
;ANSWER
yachaytech.edu.ec. 3275 IN A 190.103.191.42
;AUTHORITY
;ADDITIONAL

--- B9: Query Non-Existent Domain (nonexistdomain12345.com) ---
Caught expected NXDOMAIN Exception: The DNS query name does not exist: nonexistdomain12345.com.

Saved dnspython results to dnspython_results.json

C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop4>
```

Figure 10: CMD Terminal Screenshot: Execution of dns_part_b.py in terminal.

When executing `python dns_part_b.py`, the obtained output matched the `nslookup` tool:

```
B1 IP: ['190.103.191.42']
B2 Domain: ['dns.google.']
B3 IP: ['201.159.220.90']
B4 MX: [(0, 'yachaytech-edu-ec.mail.protection.outlook.com.')]
B5 NS: ['ns101.dominiosecuador.ec.', 'ns104.cloudns.net.', ...]
B6 SOA: Primary=ns101.cloudns.net., Serial=2026082103
```

```
B7 CNAME: ['www.microsoft.com-c-3.edgekey.net.']
B8 Debug: id 20213 opcode QUERY rcode NOERROR flags QR RD RA
B9 NXDOMAIN: The DNS query name does not exist: nonexistentdomain12345.com.
```

4 Part C: Installation, Configuration, and Testing of LDAP

The installation steps on Linux (Ubuntu/Debian) and the simulation/execution using directory scripts and LDAP authentication are documented below.

4.1 1. OpenLDAP Installation and Reconfiguration (Linux Commands)

In a Linux Debian/Ubuntu environment, the installation of the OpenLDAP server (slapd) and command-line tools (ldap-utils) is performed via:

```
sudo apt update
sudo apt install slapd ldap-utils
sudo dpkg-reconfigure slapd
```

During reconfiguration, the following parameters are specified:

1. **DNS Domain:** example.com
2. **Organization Name:** Example Inc.
3. **Administrator Password:** adminpassword

To check service status and test an initial basic search:

```
sudo systemctl status slapd
ldapsearch -x -LLL -b dc=example,dc=com
```

4.2 2. Creation of LDIF Files

Two LDIF files were created in the `Workshop4` directory to define the organizational structure and insert a user.

File 1: base.ldif (Organizational Structure):

Listing 1: Contents of base.ldif

```
dn: ou=People,dc=example,dc=com
objectClass: organizationalUnit
ou: People

dn: ou=Groups,dc=example,dc=com
```

```
objectClass: organizationalUnit
ou: Groups
```

Import command:

```
sudo ldapadd -x -D cn=admin,dc=example,dc=com -W -f base.ldif
```

File 2: user.ldif (User Insertion for Francisco Hidrobo):

Listing 2: Contents of user.ldif

```
dn: uid=francisco,ou=People,dc=example,dc=com
objectClass: inetOrgPerson
uid: francisco
sn: Hidrobo
cn: Francisco Hidrobo
userPassword: password
```

Import command:

```
sudo ldapadd -x -D cn=admin,dc=example,dc=com -W -f user.ldif
```

4.3 3. Key LDAP Operations in Python

Rather than including full script boilerplate, the core LDAP directory operations implemented in `ldap_part_c.py` using `ldap3` are detailed below:

1. OpenLDAP Connection and Administrator Binding

```
server = Server('ldap://localhost:389', get_info=ALL)
conn = Connection(server, user='cn=admin,dc=example,dc=com', password='
    adminpassword')
conn.bind()
```

Explanation: Connects to the directory server on port 389 and performs administrative binding with root DN credentials (`cn=admin,dc=example,dc=com`).

2. Directory Search (`ldapssearch`)

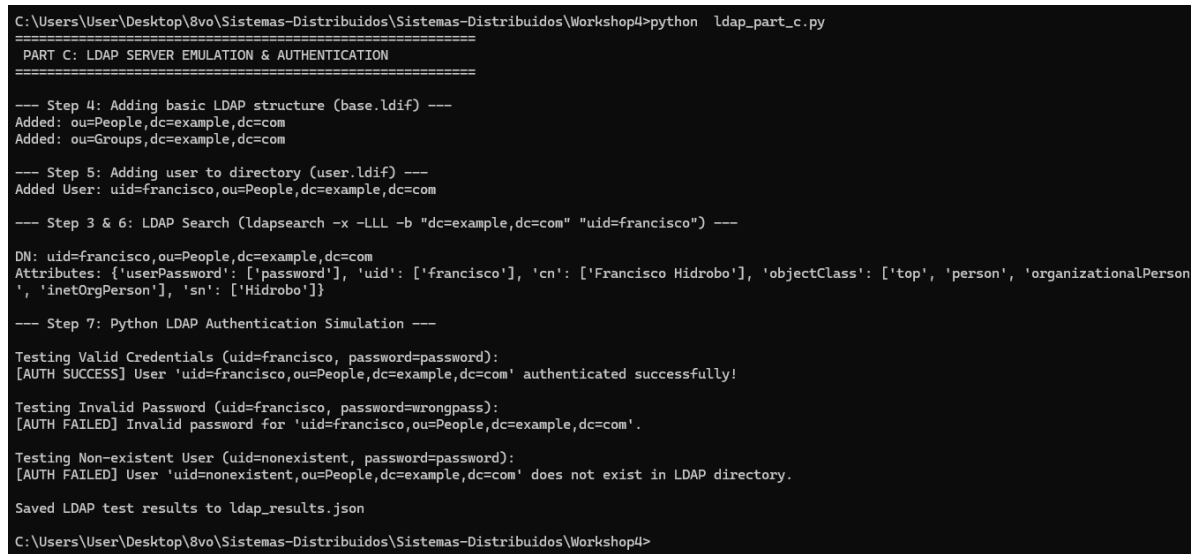
```
conn.search(
    search_base='dc=example,dc=com',
    search_filter='(uid=francisco)',
    attributes=ALL_ATTRIBUTES
)
found_dn = conn.entries[0].entry_dn
```

Explanation: Executes a subtree search starting at base DN `dc=example,dc=com` using filter `(uid=francisco)` to locate the target user entry and return its Distinguished Name.

3. User Credential Authentication

```
def authenticate_user(user_dn, password):
    conn.search(search_base='dc=example,dc=com', search_filter='(objectClass=
inetOrgPerson)', attributes=['userPassword'])
    for entry in conn.entries:
        if entry.entry_dn.lower() == user_dn.lower() and entry.userPassword[0]
== password:
            return True
    return False
```

Explanation: Verifies user identity by checking if the supplied password matches the stored userPassword attribute associated with the target DN.



```
C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop4>python ldap_part_c.py

PART C: LDAP SERVER EMULATION & AUTHENTICATION

--- Step 4: Adding basic LDAP structure (base.ldif) ---
Added: ou=People,dc=example,dc=com
Added: ou=Groups,dc=example,dc=com

--- Step 5: Adding user to directory (user.ldif) ---
Added User: uid=francisco,ou=People,dc=example,dc=com

--- Step 3 & 6: LDAP Search (ldapsearch -x -LLL -b "dc=example,dc=com" "uid=francisco") ---

DN: uid=francisco,ou=People,dc=example,dc=com
Attributes: {'userPassword': ['password'], 'uid': ['francisco'], 'cn': ['Francisco Hidrobo'], 'objectClass': ['top', 'person', 'organizationalPerson', 'inetOrgPerson'], 'sn': ['Hidrobo']}

--- Step 7: Python LDAP Authentication Simulation ---

Testing Valid Credentials (uid=francisco, password=password):
[AUTH SUCCESS] User 'uid=francisco,ou=People,dc=example,dc=com' authenticated successfully!

Testing Invalid Password (uid=francisco, password=wrongpass):
[AUTH FAILED] Invalid password for 'uid=francisco,ou=People,dc=example,dc=com'.

Testing Non-existent User (uid=nonexistent, password=password):
[AUTH FAILED] User 'uid=nonexistent,ou=People,dc=example,dc=com' does not exist in LDAP directory.

Saved LDAP test results to ldap_results.json

C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop4>
```

Figure 11: CMD Terminal Screenshot: LDAP search and authentication executed with ldap_part_c.py.

5 Part D: Special Activity - Publisher-Subscriber System with LDAP Service Discovery

In Workshop 2 (Part 2), subscribers had to know in advance the static IP addresses and ports of publishers (e.g., tcp://localhost:15001). For this workshop, a decoupled, service-oriented architecture was implemented where:

1. **LDAP Service Directory:** Endpoints are registered under the branch ou=Services,dc=example,dc=com. Each entry has the structure cn={SERVICE_NAME} and attributes such as ipHostNumber and ipServicePort.
2. **Publishers (Publisher):** Upon startup, they connect to the LDAP server and dynamically register their service name (WEATHER, FINANCE, SPORTS), IP address (127.0.0.1), and port (15001, 15002, 15003).

3. **Subscribers (Subscriber):** They do not know ports in advance. Instead, they query the LDAP server requesting the desired service. Upon receiving the response with the IP and port, they connect to the ZeroMQ socket (SUB) and filter received news topics.

5.1 Key Pub/Sub & LDAP Integration Snippets

Rather than including full script boilerplate, the core service registration and discovery routines from `pubsub_ldap.py` are highlighted below:

1. Dynamic Publisher Service Registration

```
def register_service(self, service_name, ip, port):
    dn = f"cn={service_name.upper()},ou=Services,dc=example,dc=com"
    attrs = {
        'objectClass': ['top', 'device', 'ipHost'],
        'cn': service_name.upper(),
        'ipHostNumber': str(ip),
        'ipServicePort': str(port)
    }
    self.conn.strategy.add_entry(dn, attrs)
```

Explanation: When a publisher starts up and binds to a ZeroMQ socket, it registers its endpoint in LDAP under `ou=Services,dc=example,dc=com` using standard `ipHost` attributes (`ipHostNumber` and `ipServicePort`).

2. Dynamic Subscriber Service Discovery

```
def lookup_service(self, service_name):
    search_ok = self.conn.search(
        search_base='ou=Services,dc=example,dc=com',
        search_filter=f'(cn={service_name.upper()})',
        attributes=['ipHostNumber', 'ipServicePort']
    )
    if search_ok and self.conn.entries:
        entry = self.conn.entries[0]
        return entry.ipHostNumber[0], int(entry.ipServicePort[0])
    return None, None
```

Explanation: Subscribers query the central LDAP registry for a desired service name (e.g., `WEATHER`), dynamically discovering the active IP and port to connect their ZeroMQ SUB socket without hardcoded addresses.

5.2 Live Execution Test and Screenshot

```
Saved Pub/Sub LDAP execution results to pubsub_ldap_results.json

C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop4>python pubsub_ldap.py

=====
PART D: SPECIAL ACTIVITY - PUB/SUB WITH LDAP DISCOVERY
=====

1. Starting Publishers & Registering Services in LDAP...
[LDAP REGISTRY] Registered service 'WEATHER' -> Host: 127.0.0.1, Port: 15001
[LDAP REGISTRY] Registered service 'FINANCE' -> Host: 127.0.0.1, Port: 15002
[LDAP REGISTRY] Registered service 'SPORTS' -> Host: 127.0.0.1, Port: 15003

2. Starting Subscriber with LDAP Dynamic Discovery...

--- [Subscriber-LDAP-Client] QUERYING LDAP DIRECTORY FOR SERVICES: ['WEATHER', 'FINANCE', 'SPORTS'] ---
[LDAP DISCOVERY] Resolved service 'WEATHER' from LDAP -> 127.0.0.1:15001
[Subscriber-LDAP-Client] Subscribed to topic 'WEATHER' at tcp://127.0.0.1:15001
[LDAP DISCOVERY] Resolved service 'FINANCE' from LDAP -> 127.0.0.1:15002
[Subscriber-LDAP-Client] Subscribed to topic 'FINANCE' at tcp://127.0.0.1:15002
[LDAP DISCOVERY] Resolved service 'SPORTS' from LDAP -> 127.0.0.1:15003
[Subscriber-LDAP-Client] Subscribed to topic 'SPORTS' at tcp://127.0.0.1:15003
[Subscriber-LDAP-Client] Listening for messages on endpoints: ['tcp://127.0.0.1:15001', 'tcp://127.0.0.1:15002', 'tcp://127.0.0.1:15003']

[PUB-WEATHER] Sent: WEATHER [21:33:26] #1 Guayaquil: 28.0C, Soleado
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:26] #1 Guayaquil: 28.0C, Soleado
[PUB-FINANCE] Sent: FINANCE [21:33:26] #1 AAPL: 225.50
[Subscriber-LDAP-Client RECIBIDO] FINANCE [21:33:26] #1 AAPL: 225.50
[PUB-SPORTS] Sent: SPORTS [21:33:26] #1 LigaPro: LDU 2 - 1 BSC
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:26] #1 LigaPro: LDU 2 - 1 BSC
[PUB-WEATHER] Sent: WEATHER [21:33:27] #2 Guayaquil: 28.0C, Soleado
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:27] #2 Guayaquil: 28.0C, Soleado
[PUB-FINANCE] Sent: FINANCE [21:33:27] #2 USD/EUR: 0.92
[Subscriber-LDAP-Client RECIBIDO] FINANCE [21:33:27] #2 USD/EUR: 0.92
[PUB-SPORTS] Sent: SPORTS [21:33:27] #2 Champions: Real Madrid 3 - 1 Bayern
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:27] #2 Champions: Real Madrid 3 - 1 Bayern

[PUB-SPORTS] Sent: SPORTS [21:33:28] #3 LigaPro: LDU 2 - 1 BSC
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:28] #3 LigaPro: LDU 2 - 1 BSC
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:29] #4 Quito: 18.5C, Lluvia
[PUB-WEATHER] Sent: WEATHER [21:33:29] #4 Quito: 18.5C, Lluvia
[PUB-FINANCE] Sent: FINANCE [21:33:29] #4 BTC/USD: 64500.0
[Subscriber-LDAP-Client RECIBIDO] FINANCE [21:33:29] #4 BTC/USD: 64500.0
[PUB-SPORTS] Sent: SPORTS [21:33:29] #4 LigaPro: LDU 2 - 1 BSC
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:29] #4 LigaPro: LDU 2 - 1 BSC
[PUB-WEATHER] Sent: WEATHER [21:33:30] #5 Quito: 18.5C, Lluvia
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:30] #5 Quito: 18.5C, Lluvia
[PUB-FINANCE] Sent: FINANCE [21:33:30] #5 USD/EUR: 0.92
[Subscriber-LDAP-Client RECIBIDO] FINANCE [21:33:30] #5 USD/EUR: 0.92
[PUB-SPORTS] Sent: SPORTS [21:33:30] #5 Champions: Real Madrid 3 - 1 Bayern
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:30] #5 Champions: Real Madrid 3 - 1 Bayern
[PUB-WEATHER] Sent: WEATHER [21:33:31] #6 Guayaquil: 28.0C, Soleado
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:31] #6 Guayaquil: 28.0C, Soleado
[PUB-FINANCE] Sent: FINANCE [21:33:31] #6 AAPL: 225.50
[Subscriber-LDAP-Client RECIBIDO] FINANCE [21:33:31] #6 AAPL: 225.50
[PUB-SPORTS] Sent: SPORTS [21:33:31] #6 LigaPro: LDU 2 - 1 BSC
[Subscriber-LDAP-Client RECIBIDO] SPORTS [21:33:31] #6 LigaPro: LDU 2 - 1 BSC

3. Stopping Pub/Sub System...
[PUB-WEATHER] Sent: WEATHER [21:33:32] #7 Guayaquil: 28.0C, Soleado
[Subscriber-LDAP-Client RECIBIDO] WEATHER [21:33:32] #7 Guayaquil: 28.0C, Soleado
[PUB-WEATHER] Stopped.
[Subscriber-LDAP-Client] Stopped.
[PUB-FINANCE] Sent: FINANCE [21:33:32] #7 USD/EUR: 0.92
[PUB-FINANCE] Stopped.
[PUB-SPORTS] Sent: SPORTS [21:33:32] #7 LigaPro: LDU 2 - 1 BSC
[PUB-SPORTS] Stopped.

System Execution Completed! Total Messages Received by Subscriber: 19
Saved Pub/Sub LDAP execution results to pubsub_ldap_results.json

C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop4>
```

Figure 12: CMD Terminal Screenshot: Complete execution of the Pub/Sub system with live LDAP registration and discovery.

6 Conclusions

1. **DNS:** The use of `nslookup` and `dnspython` enabled first-hand verification of DNS distributed architecture. The utility of record types such as A (IPv4), PTR (Reverse Resolution), MX (Mail Routing), NS (Authoritative Servers), and SOA (Zone Parameters) was confirmed, as well as error handling via NXDOMAIN responses.
2. **LDAP:** LDAP provides a highly efficient hierarchical structure for storing organizational and infrastructure information. LDIF files facilitate reproducible provisioning of organizational units and user accounts.

3. **Service Discovery:** Integrating LDAP with the Publisher-Subscriber pattern demonstrates the principle of location decoupling in distributed systems. Subscribers do not require static knowledge of server locations; instead, they query a centralized directory service that grants mobility and tolerance to network changes.